Extending Hierarchical Reinforcement Learning to Digital Timed Reward Machines

Rayan Guergueb

Abstract

Timed Reward Machines (TRMs) extend Reward Machines (RMs) with explicit temporal constraints and delay-aware rewards, enabling the specification of tasks that require an agent not only to satisfy logical objectives but also to do so at appropriate times. The original TRM framework provides digital-time and real-time interpretations together with counterfactual imagining (CI), but it does not exploit the hierarchical decomposition mechanisms that have proven effective for classical RMs. This report studies a hierarchical extension of the digital-time TRM setting inspired by Hierarchical Reinforcement Learning for Reward Machines (HRM). Our objective is to test whether the logical-temporal structure exposed by a TRM can be leveraged to learn option-based policies that improve learning efficiency while remaining faithful to the digital semantics used in the original TRM work.

We formulate a digital-time TRM model with a HRM extension, in which the high-level controller selects a target TRM state and the low-level controller executes augmented actions composed of a delay and a primitive action. The extension preserves the digital TRM product structure and incorporates local HRM-style shaping rewards for option learning. In addition, we define an algorithmic combination of this hierarchical model with the local counterfactual imagining mechanism used in the TRM paper. We describe the full mathematical model, the resulting learning algorithm, the experimental protocol, and the implementation assumptions used in our study.

1 Introduction

Reward Machines (RMs) provide a compact automata-based representation of non-Markovian reward structure by augmenting an environment with a symbolic state that tracks progress through a task specification. Prior work showed that the RM structure can be exploited algorithmically in several ways, notably through counterfactual experience sharing and hierarchical decomposition into options. In particular, Hierarchical Reinforcement Learning for Reward Machines (HRM) learns one option for each relevant transition between RM states and a high-level policy that composes such options into a complete task-solving strategy.

Timed Reward Machines (TRMs) generalize this formalism by incorporating clocks, timing guards, and delay-aware rewards. The TRM framework introduces digital-time and real-time semantics and proposes model-free learning procedures based on cross-products between the environment and the timed reward specification. In the digital-time setting, actions are augmented with delays and learning takes place on a finite product abstraction. The TRM paper also rely on Counterfactual Imagining (CI), a local counterfactual mechanism introduced in the prior Reward Machines works, that improves sample efficiency. It was adapted to TRM by replaying an observed transition under nearby clock valuations and alternative delays.

Despite these advances, the original TRM framework does not exploit hierarchical structure in the sense of HRM. This leaves open a natural question: can the logical-temporal structure encoded

by a digital TRM be exploited to define meaningful options, in the same spirit as HRM for classical RMs, while respecting the timed semantics of the task?

This question is particularly interesting because, in the classical RM setting, HRM did not always yield consistently strong empirical improvements. While the hierarchical decomposition is conceptually appealing, its practical benefit may remain limited when the underlying product structure is still relatively simple. In the timed setting, however, the learning problem becomes significantly more complex: the digital TRM product augments the environment not only with a symbolic state, but also with a bounded clock valuation, and the action space itself is extended with delays. This additional structure increases the dimensionality and difficulty of the control problem, making hierarchical decomposition a more compelling candidate for improving learning. From this perspective, HRM may yield stronger results in the TRM setting, which is richer and more demanding than the classical RM setting.

This study deliberately focuses on the digital-time setting. The digital abstraction is finite, algorithmically tractable, and already serves as one of the main baselines in the TRM framework. Moreover, restricting attention to digital time allows us to isolate the contribution of hierarchical decomposition without conflating it with the additional technical complexity of real-time abstractions such as corner-point methods.

Research objective. The purpose of this work is to extend the digital-time baseline with a hierarchical mechanism that is as faithful as possible to HRM for RMs. Concretely, we aim to answer the following methodological question:

> Can a digital-time TRM be augmented through HRM, using target TRM states as subgoals and augmented actions as low-level controls, while preserving the product semantics and allowing a fair comparison against the original digital-time TRM baselines?

2 Scope and Positioning

What had not been done (to our knowledge). Before this project, the TRM setting did not include an explicit HRM-style hierarchical decomposition over digital timed products. In particular, we did not find a prior implementation that jointly combines:

(i) option-based hierarchy over digital TRM product states, and

(ii) local timed CI integrated into that hierarchy.

Our position relative to prior work. This work is therefore a targeted extension of the digital TRM baseline:

- we keep the original TRM product semantics and digital action model;
- we add HRM-style high-level target selection and low-level option learning;
- we evaluate the hierarchy both with and without local CI under the same protocol.

What we set out to show. The empirical goal of the project is to compare four conditions under a common experimental protocol:

(i) the digital-time TRM baseline (TRM vanilla),

(ii) the digital-time TRM baseline augmented with local counterfactual imagining (TRM+CI),

(iii) the proposed hierarchical extension (TRM+HRM), and

(iv) the combination of the hierarchical extension with local CI (TRM+HRM+CI).

3 Background

3.1 Markov decision processes

We consider a labeled Markov decision process (MDP)

$$\mathcal{M} = (S, A, P, \gamma, AP, L),$$

where S is a finite set of environment states, A is a finite set of primitive actions, $P(s' \mid s, a)$ is the transition probability, $\gamma \in (0, 1)$ is the discount factor, AP is a finite set of atomic propositions, and $L : S \times A \times S \to 2^{AP}$ is a labeling function. As in the TRM framework, the labels provide the symbolic observations used by the reward specification.

3.2 Reward Machines

A Reward Machine (RM) is a finite automaton that augments the environment with a symbolic state tracking progress through a task. The product MDP induced by an RM and an environment transforms a non-Markovian reward problem into a Markovian one over states (s, u), where u denotes the current RM state. Prior work showed that this structure can be exploited by counterfactual experience sharing (CRM) and by hierarchical decomposition (HRM).

3.3 Timed Reward Machines

Timed Reward Machines extend classical reward machines with clocks and timing guards. A TRM specification is able to express both the logical ordering of events and the time intervals in which they are allowed or rewarded. The TRM formalism considered follows the digital-time interpretation introduced in the original TRM paper.

A timed reward machine is written as

$$\mathcal{A} = (U, u_0, F, \Delta_u, \Delta_r, X),$$

where:

- U is a finite set of non-terminal TRM states,
- $u_0 \in U$ is the initial TRM state,
- F is a set of terminal TRM states,
- X is a finite set of clocks,
- $\Delta_u \subseteq U \times 2^{AP} \times \Phi(X) \times 2^X \times (U \cup F)$ is the timed transition relation of the TRM,
- $\Delta_r = \Delta_u^r \cup \Delta_\theta^r$ contains a state-based reward component and a transition-based reward component.

A transition in Δ_u is denoted by :

$$\theta = (u, \ell, \phi, \rho, u'),$$

meaning that from state u, after observing label ℓ, if guard ϕ is satisfied, the TRM can move to state u' while resetting the clocks in ρ.

3.4 Digital-time semantics

In the digital-time setting, time takes values in $\mathbb{N}$ and a decision consists of choosing an augmented action of the form (d, a), where d is a non-negative delay and $a \in A$ is a primitive environment action. Intuitively, the agent waits for d units of time and then executes a, which consumes one additional unit of time. Hence a single augmented action consumes $d + 1$ units of digital time.

Let $M_{\max}$ denote the largest constant appearing in any timing guard of the TRM. Following the digital abstraction used in the TRM paper, we define a bounded valuation space

$$\mathcal{V} = \left(\{0, 1, \ldots, M_{\max}\} \cup \{\infty\}\right)^{|X|}$$

and a finite set of delays

$$\mathcal{D} = \{0, 1, \ldots, M_{\max}\}.$$

The digital product state is therefore of the form (s, u, v), where $v \in \mathcal{V}$ is the bounded clock valuation.

4 Problem Statement

The original digital-time TRM baseline learns directly on the timed product using tabular Q-learning over augmented actions. The question we address is whether this same product can be endowed with a useful hierarchical decomposition analogous to HRM for classical RMs.

Formally, our goal is to extend the digital-time TRM setting with a hierarchy consisting of:

- a *high-level policy* that selects which logical-temporal progress state to pursue next, and
- a collection of *low-level option policies* that select augmented actions (d, a) so as to realize the chosen progress.

5 Digital TRM Product Construction

We now define the product state space used by all four compared methods.

5.1 Product states and augmented actions

The digital-time TRM product is defined by

$$S^{\otimes} = S \times U \times \mathcal{V}, \qquad A^{\otimes} = \mathcal{D} \times A.$$

A product state is denoted by

$$s^{\otimes} = (s, u, v) \in S^{\otimes},$$

and an augmented action by

$$a^{\otimes} = (d, a) \in A^{\otimes}.$$

5.2 Valuation update

For a clock valuation $v \in \mathcal{V}$ and delay $d \in \mathcal{D}$, let $v \oplus d$ denote the bounded valuation obtained after the elapsed time $d + 1$. Each component is incremented by $d + 1$ and then truncated to ∞ when exceeding the maximal representable guard constant $M_{\max}$. If a transition resets a clock subset $\rho \subseteq X$, the post-transition valuation is denoted by $[\rho](v \oplus d)$.

5.3 Timed product transition

Given a product state $s^{\otimes} = (s, u, v)$, an augmented action $a^{\otimes} = (d, a)$, and a next product state $s' \sim P(\cdot \mid s, a)$, let

$$\ell = L(s, a, s').$$

We search for an enabled outgoing TRM transition

$$\theta = (u, \ell, \phi, \rho, u') \in \Delta_u$$

such that $v \oplus d \models \phi$. If such a transition exists, the next TRM state is u' and the new valuation is

$$v' = [\rho](v \oplus d).$$

If no outgoing TRM transition is enabled, the implementation treats the event as an unspecified or unintended behavior, and sends the trajectory to a terminal failure outcome with an associated penalty. This convention makes it possible to penalize executions that fall outside the part of the task explicitly modeled by the TRM, in particular when the specification is not fully defined.

5.4 Reward

The product reward follows the digital TRM semantics from the original framework. It combines two contributions inherited from the TRM reward structure: a state-based reward accumulated during the chosen delay, and a transition-based reward attached to the TRM transition that is triggered. Formally, for a realized product transition :

$$(s, u, v) \xrightarrow{(d,a)} (s', u', v'),$$

such that there exists an enabled TRM transition :

$$\theta = (u, L(s, a, s'), \phi, \rho, u') \in \Delta_u,$$

we write the immediate reward as

$$R^{\otimes}\big((s, u, v), (d, a), (s', u', v')\big) = r_u + r_\theta,$$

where

$$r_u = \frac{1 - \gamma^d}{1 - \gamma}\, \Delta_u^r(u), \qquad r_\theta = \Delta_\theta^r(\theta).$$

Hence, r_u represents the state-based reward accumulated while the agent remains in TRM state u during the delay period d, whereas r_θ is the reward associated with the TRM transition actually taken toward u'.

6 Mathematical Extension: HRM for Digital TRMs

6.1 Available high-level objectives

For each TRM state $u \in U$, define its successor set :

$$\mathrm{Succ}(u) = \{\tilde{u} \in U \cup F \mid \exists\, (u, \ell, \phi, \rho, \tilde{u}) \in \Delta_u\}.$$

Under the target-state indexing assumption, the set of high-level options available at source state u is

$$\mathcal{O}(u) = \{o_{u,\tilde{u}} \mid \tilde{u} \in \mathrm{Succ}(u),\ \tilde{u} \neq u\}.$$

Thus the high-level controller does not choose a syntactic TRM transition, but rather a target TRM state $\tilde{u}$ corresponding to a meaningful logical-temporal progress objective.

6.2 Option definition

The hierarchy operates as follows. When the agent is in a product state (s, u, v) and no option is currently active, the high-level controller selects a symbolic-temporal target state $\tilde{u} \in \mathrm{Succ}(u) \setminus \{u\}$. This choice activates an option whose role is to reach the intended TRM state $\tilde{u}$. Once the option is active, control is transferred to the low level, which selects augmented actions of the form (d, a) while conditioning on the current environment state and clock valuation. The option remains in control as long as the TRM stays in its current source state u. As soon as the TRM state changes, or the episode terminates, control is returned to the high-level policy, which may then select a new symbolic target.

For each pair $(u, \tilde{u})$ with $\tilde{u} \in \mathrm{Succ}(u)$ and $\tilde{u} \neq u$, we define an option

$$o_{u,\tilde{u}} = \langle I_{u,\tilde{u}}, \pi_{u,\tilde{u}}, \beta_{u,\tilde{u}} \rangle.$$

Initiation set. The option may be started whenever the current TRM state equals u:

$$I_{u,\tilde{u}} = \{(s, u, v) \in S^{\otimes} \mid s \in S,\ v \in \mathcal{V}\}.$$

Low-level policy. Once the option is activated, the low-level policy selects augmented actions:

$$\pi_{u,\tilde{u}}(a^{\otimes} \mid s, u, v).$$

Since the source TRM state u remains fixed while the option is active, we may equivalently write

$$\pi_{u,\tilde{u}}(a^{\otimes} \mid s, v).$$

Termination condition. The option terminates as soon as the TRM state changes or the episode ends:

$$\beta_{u,\tilde{u}}(s', u', v') = \begin{cases} 1 & \text{if } u' \neq u \text{ or the episode terminates,} \\ 0 & \text{otherwise.} \end{cases}$$

In other words, the option retains control while the symbolic source state remains unchanged, and releases control as soon as the execution leaves that state. This is the proposed timed analogue of HRM for classical RMs.

6.3 High-level value function

The high-level controller learns an action-value function

$$Q^{\mathrm{H}}(s, u, v, \tilde{u}), \qquad \tilde{u} \in \mathrm{Succ}(u) \setminus \{u\},$$

that estimates the return obtained by launching option $o_{u,\tilde{u}}$ from the product state (s, u, v). The high-level policy is implemented as an ε-greedy choice over valid targets:

$$\mu(\tilde{u} \mid s, u, v) \in \mathrm{Greedy}_{\varepsilon}(Q^{\mathrm{H}}(s, u, v, \cdot)).$$

6.4 Low-level value functions

For each option $o_{u,\tilde{u}}$, the low-level learner maintains a separate action-value function

$$Q^{\mathrm{L}}_{u,\tilde{u}}(s, v, d, a),$$

which estimates the value of executing augmented action $a^{\otimes} = (d, a)$ while pursuing target state $\tilde{u}$ from source state u. The low-level behavior policy associated with option $o_{u,\tilde{u}}$ is implemented as an ε-greedy policy over augmented actions:

$$\pi_{u,\tilde{u}}(a^{\otimes} \mid s, u, v) \in \mathrm{Greedy}_{\varepsilon}\big(Q^{\mathrm{L}}_{u,\tilde{u}}(s, v, \cdot, \cdot)\big).$$

6.5 Local HRM shaping reward

As in HRM for classical RMs, each option is trained with an option-specific shaped product reward. For a realized low-level transition

$$(s, u, v) \xrightarrow{(d,a)} (s', u', v'),$$

we define

$$R^{\otimes}_{u,\tilde{u}}\big((s, u, v), (d, a), (s', u', v')\big) = R^{\otimes}\big((s, u, v), (d, a), (s', u', v')\big) + r_{u,\tilde{u}}(u'),$$

where

$$r_{u,\tilde{u}}(u') = \begin{cases} r_+ & \text{if } u' = \tilde{u}, \\ r_- & \text{if } u' \notin \{u, \tilde{u}\}, \\ 0 & \text{if } u' = u. \end{cases}$$

The additional term $r_{u,\tilde{u}}(u')$ is an option-specific shaping signal. It becomes informative only when the transition changes the current TRM state, and evaluates whether that symbolic transition corresponds to the intended target $\tilde{u}$. By default, our implementation uses $r_+ = 1.0$ and $r_- = 0.0$, matching the common HRM convention used in RM experiments.

6.6 Low-level update rule

Let

$$s^{\otimes} = (s, u, v), \qquad a^{\otimes} = (d, a), \qquad s^{\otimes\prime} = (s', u', v').$$

The low-level Q-learning target for option $o_{u,\tilde{u}}$ is

$$y^{\mathrm{L}}_{u,\tilde{u}} = R^{\otimes}_{u,\tilde{u}}(s^{\otimes}, a^{\otimes}, s^{\otimes\prime}) + \big(1 - \beta_{u,\tilde{u}}(s^{\otimes\prime})\big)\gamma^{d+1} \max_{(d',a') \in \mathcal{D} \times A} Q^{\mathrm{L}}_{u,\tilde{u}}(s', v', d', a').$$

The factor $(1 - \beta_{u,\tilde{u}}(s^{\otimes\prime}))$ ensures that the bootstrap term is retained only when the realized transition keeps the option active. If $\beta_{u,\tilde{u}}(s^{\otimes\prime}) = 1$, the option terminates and the target reduces to the immediate shaped reward. The update is then:

$$Q^{\mathrm{L}}_{u,\tilde{u}}(s, v, d, a) \leftarrow Q^{\mathrm{L}}_{u,\tilde{u}}(s, v, d, a) + \alpha_{\mathrm{L}}\Big(y^{\mathrm{L}}_{u,\tilde{u}} - Q^{\mathrm{L}}_{u,\tilde{u}}(s, v, d, a)\Big).$$

6.7 High-level update

In the previous sections, we used unindexed notation for generic product transitions. Here, however, the update is defined over an entire option execution, and it is therefore important to revert to the trajectory indexing convention of the digital TRM framework.

We index low-level decisions by i, where i denotes the step index in the trajectory. The indexing starts at 0, and between steps i and $i+1$ there elapses $d_i + 1$ units of digital time: d_i units for the chosen delay and 1 unit for the execution of the primitive action. Thus, i is not a time variable, but an index in the sequence of low-level decisions.

We consider an option $o_{u_i,\tilde{u}}$ initiated at step i in product state

$$s_i^{\otimes} = (s_i, u_i, v_i),$$

and terminating after $K \geq 1$ low-level decisions in state

$$s_{i+K}^{\otimes} = (s_{i+K}, u_{i+K}, v_{i+K}).$$

Here, K denotes the number of low-level decisions executed while the option remains active. Accordingly, $i+K$ is the index of the step at which the option terminates.

To simplify the notation, we introduce the partial elapsed duration

$$\tau_{i,j} = \sum_{m=0}^{j-1} (d_{i+m} + 1), \qquad j \geq 0,$$

with the convention

$$\tau_{i,0} = 0.$$

The quantity $\tau_{i,j}$ denotes the total digital time elapsed between the beginning of the option at step i and its j-th low-level decision. In particular, the total elapsed digital duration of the option is

$$\tau = \tau_{i,K} = \sum_{j=0}^{K-1} (d_{i+j} + 1).$$

This quantity measures how much digital time elapses during the option execution. The convention $\tau_{i,0} = 0$ corresponds to the fact that no time has elapsed yet before the first low-level decision of the option.

Let the discounted reward accumulated during the execution of this option be

$$G_i = \sum_{j=0}^{K-1} \gamma^{\tau_{i,j}} R^{\otimes}(s_{i+j}^{\otimes}, (d_{i+j}, a_{i+j}), s_{i+j+1}^{\otimes}).$$

G_i is the true discounted return accumulated over the whole option execution.

The high-level target is then

$$y_i^{\mathrm{H}} = G_i + \mathbf{1}\{s_{i+K}^{\otimes} \text{ is non-terminal and } u_{i+K} \notin F\}\gamma^{\tau} \max_{\bar{u} \in \mathrm{Succ}(u_{i+K}) \setminus \{u_{i+K}\}} Q^{\mathrm{H}}(s_{i+K}, u_{i+K}, v_{i+K}, \bar{u}).$$

The indicator function ensures that the bootstrap term is included only when the option terminates in a non-terminal product state from which a new high-level choice is still possible. The discount factor γ^{τ} accounts for the fact that the option may consume a variable amount of digital time before control returns to the high level.

The corresponding update is

$$Q^{\mathrm{H}}(s_i, u_i, v_i, \tilde{u}) \leftarrow Q^{\mathrm{H}}(s_i, u_i, v_i, \tilde{u}) + \alpha_{\mathrm{H}}\Big(y_i^{\mathrm{H}} - Q^{\mathrm{H}}(s_i, u_i, v_i, \tilde{u})\Big).$$

7 Algorithmic Extension with Local Counterfactual Imagining

7.1 Motivation

A naive extension of CRM to digital-time TRMs would require sharing experience across a state space that includes both symbolic TRM states and bounded clock valuations. Such an exhaustive counterfactual construction would be prohibitively expensive because the digital product already expands the state space into $S \times U \times \mathcal{V}$ and the action space into $\mathcal{D} \times A$. The TRM paper therefore introduces a local alternative, Counterfactual Imagining (CI), which varies only nearby clock valuations and alternative delays rather than the full product state.

We adopt that same philosophy here. CI is used as a local low-level augmentation mechanism, not as a full exhaustive counterfactual cross-product.

7.2 Observed real transition

Consider a real low-level transition

$$((s, u, v), (d, a), r, (s', u', v')).$$

In the CI construction, the environment states s and s', the primitive action a, and the source TRM state u are kept fixed. Only the clock valuation and delay are varied.

7.3 Candidate generation

For a real transition, the CI generator constructs alternative pairs $(\bar{v}, \bar{d})$ according to the following rules:

(1) $\bar{v}$ must lie within an L_∞ ball of radius r_{crm} around the observed valuation v;

(2) $\bar{d}$ must belong to the discrete delay grid allowed by the digital abstraction;

(3) the pair $(\bar{v}, \bar{d})$ is kept only if at least one relevant outgoing guard from source state u is satisfiable for the observed label $L(s, a, s')$.

7.4 Filtering and reuse

To control combinatorial growth, at most `crm_nums=15` counterfactual transitions are retained per real transition, ranked by immediate product reward. Deduplication is performed at the implementation level using the counterfactual observation and action. If no relevant guard is satisfiable for the observed label, no CI transitions are generated.

7.5 Interaction with the hierarchy

In the hierarchical setting, CI transitions are used only for low-level learning. The high-level semi-Markov target is always computed from the real option trajectory actually executed in the environment. This choice isolates the effect of local counterfactual low-level credit assignment without distorting the high-level option selection problem.

Algorithm 1 Digital TRM-HRM with optional local CI

Initialize Q^{H} and all low-level tables $\{Q^{\mathrm{L}}_{u,\tilde{u}}\}$
for each episode **do**
  Reset environment state s, TRM state $u \leftarrow u_0$, valuation $v \leftarrow \mathbf{0}$
  Set current option to `None`
  Initialize option-level accumulators
  while episode not terminated **do**
    if no option is active and $\mathrm{Succ}(u) \setminus \{u\} \neq \emptyset$ **then**
      Select target $\tilde{u} \sim \varepsilon$-greedy$(Q^{\mathrm{H}}(s, u, v, \cdot))$
      Activate option $o_{u,\tilde{u}}$
      Initialize high-level accumulators for that option
    end if
    Select augmented action (d, a) using ε-greedy$(Q^{\mathrm{L}}_{u,\tilde{u}}(s, v, \cdot, \cdot))$
    Sample $s' \sim P(\cdot \mid s, a)$
    Compute label $\ell = L(s, a, s')$
    Compute TRM successor (u', v') and product reward $R^{\otimes}((s, u, v), (d, a), (s', u', v'))$
    Update the active low-level option, or all outgoing options from source u if parallel updates are enabled
    if CI is enabled **then**
      Generate local counterfactual transitions around (v, d)
      Apply the same low-level updates to retained CI transitions
    end if
    Accumulate real reward and elapsed time for the active option
    if $u' \neq u$ or episode terminates **then**
      Perform high-level semi-Markov update for the active option
      Deactivate the option
    end if
    $(s, u, v) \leftarrow (s', u', v')$
  end while
end for

8 Experimental Protocol

8.1 Benchmarks

All experiments are conducted on the digital-time tasks from the TRM repository in the `Taxi` and `Frozen_Lake` environments. We use the same digital abstraction family and the same TRM specification file (`disc_vs_cont.txt`) across the compared methods.

8.2 Compared conditions

The four compared methods are:

(1) **TRM vanilla**: digital-time TRM baseline with tabular Q-learning over the product and augmented actions.

(2) **TRM+CI**: the same baseline augmented with the local counterfactual imagining mechanism.

(3) **TRM+HRM**: the proposed hierarchical extension without counterfactual imagining.

(4) **TRM+HRM+CI**: the proposed hierarchical extension with local counterfactual imagining used at the low level.

8.3 Metrics

The experimental evaluation follows the reporting style of the TRM paper. We record at least the following metrics:

(i) discounted return during training and evaluation, and

(ii) episode time, including delays.

These metrics are particularly informative in timed tasks: a method may complete an episode quickly while violating timing constraints, or may obtain higher reward precisely because it learns to wait when necessary.

9 Results

We compare four digital-time conditions under a common protocol: *TRM vanilla*, *TRM+CI*, *TRM+HRM*, and *TRM+HRM+CI*. All runs use 10 seeds (42–51), 300 000 timesteps, and identical evaluation and logging settings.

9.1 Quantitative summary at final training step

Table 1: Final evaluation performance at global step 300 000 (mean +/- std over 10 seeds).

Task	Method	Discounted return	Episode time
Taxi	TRM vanilla	1296.07 +/- 39.95	50.0 +/- 2.28
	TRM+CI	1489.63 +/- 19.14	40.2 +/- 1.40
	TRM+HRM	1303.93 +/- 34.39	49.9 +/- 2.91
	TRM+HRM+CI	**1536.35 +/- 13.53**	**36.8 +/- 0.98**
Frozen Lake	TRM vanilla	**240.43 +/- 425.80**	**40.6 +/- 25.39**
	TRM+CI	171.54 +/- 433.60	44.3 +/- 24.46
	TRM+HRM	94.79 +/- 397.84	53.9 +/- 27.35
	TRM+HRM+CI	167.94 +/- 540.97	40.7 +/- 23.59

9.2 Taxi

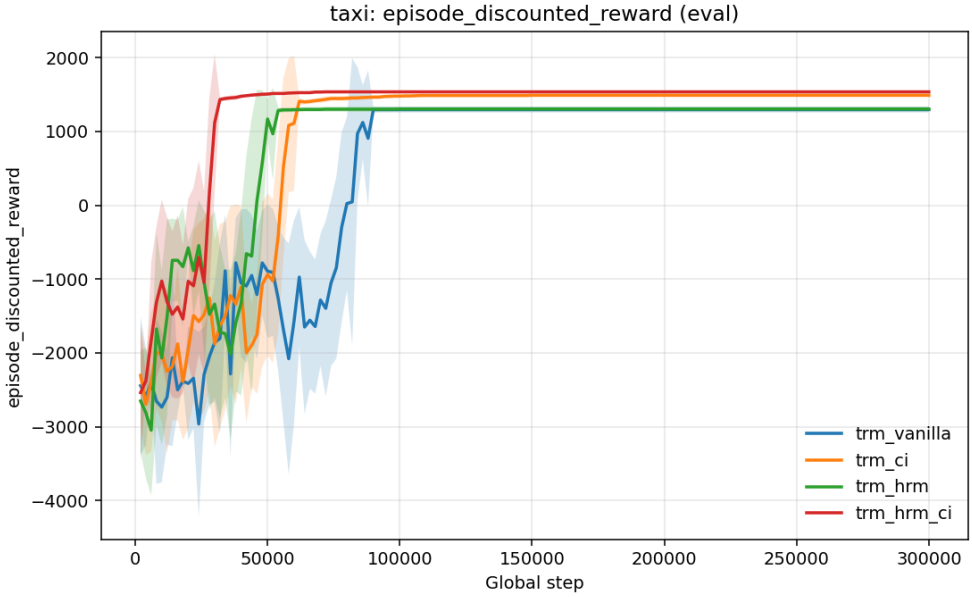

Figure 1: Evaluation discounted return on the digital-time Taxi task.

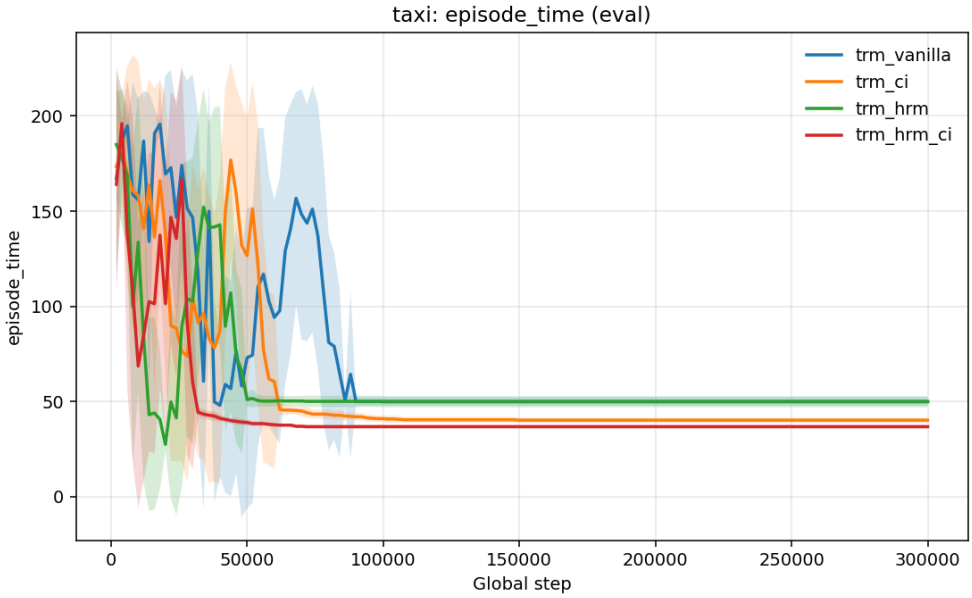

Figure 2: Evaluation episode time on the digital-time Taxi task.

The Taxi domain provides the strongest evidence in favor of the proposed extension. Relative to *TRM vanilla*, the CI baseline (*TRM+CI*) already yields a substantial improvement, both in final discounted return and in episode-time efficiency. More precisely, it improves final return by about 14.9% and reduces episode time by about 19.6%. This result is consistent with the original TRM motivation for local counterfactual reuse.

The hierarchy-only variant (*TRM+HRM*) behaves differently. At the final training step, it remains very close to the vanilla baseline, with only marginal differences in both return and episode time. This suggests that hierarchy alone does not automatically guarantee a substantial improvement, at least under the present hyperparameter setting and training budget.

The most important result on Taxi is the performance of the combined method *TRM+HRM+CI*. This variant achieves the best final score on both reported metrics. Compared with the vanilla baseline, it improves discounted return by about 18.5% and reduces episode time by about 26.4%. Even compared with the already strong *TRM+CI* baseline, it still yields an additional gain of roughly 3.1% in return and 8.5% in episode-time efficiency. In other words, on Taxi, hierarchy and local CI are not redundant: their combination produces the strongest outcome.

The learning curves also suggest an advantage in convergence speed. In the aggregated return curve, *TRM+HRM+CI* reaches a return of 1400 around step 32 000, whereas *TRM+CI* reaches the same level only around step 62 000. Moreover, *TRM+HRM+CI* is the only method that reaches a return of 1500, doing so around step 48 000. These observations reinforce the conclusion that, in Taxi, the proposed hierarchical extension is not only beneficial in final performance but also accelerates learning when combined with CI.

9.3 Frozen Lake

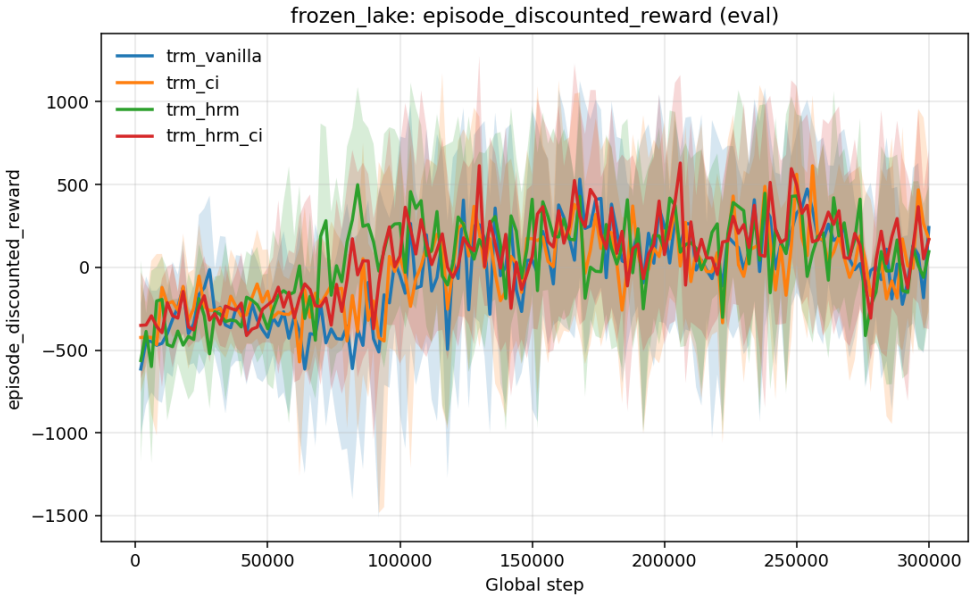

Figure 3: Evaluation discounted return on the digital-time Frozen Lake task.

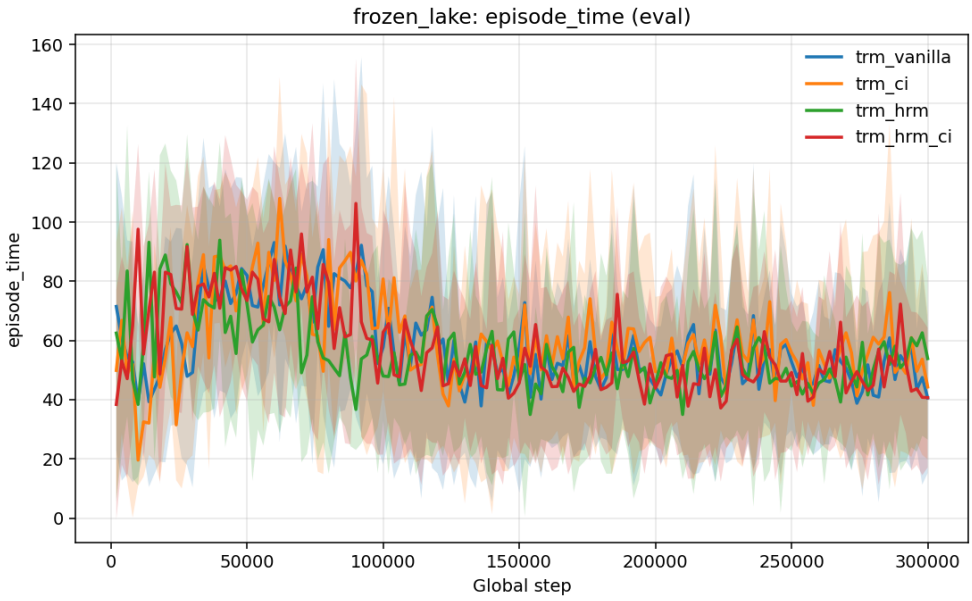

Figure 4: Evaluation episode time on the digital-time Frozen Lake task.

Frozen Lake leads to a much less decisive picture. Although the final-step table appears to rank *TRM vanilla* first on both reported metrics, this ordering should be interpreted very cautiously. The standard deviations are extremely large, especially for discounted return, where they systematically exceed 390 and in several cases are larger than the corresponding means themselves. This indicates strong sensitivity to random seeds and substantial stochastic variability.

Under such variance, differences between final-step means are not sufficient to support strong conclusions about method superiority. The practical conclusion on Frozen Lake is therefore not that one method clearly dominates, but rather that the present experimental budget is insufficient to establish a robust ranking. A more convincing analysis on this domain would require either more seeds, longer training horizons, or statistical comparisons performed over full learning curves rather than final values alone.

9.4 Comparison against baselines and prior expectations

Taken together, these results allow us to answer the central comparison questions of the project. Relative to the digital TRM baseline, the combination of hierarchy and local CI clearly improves performance on Taxi. Relative to the already strengthened *TRM+CI* baseline, the hierarchical extension still yields additional gains on that domain. At the same time, the results also confirm that hierarchy is not uniformly beneficial across tasks: unlike Taxi, Frozen Lake does not exhibit a similarly clear advantage under the current protocol. Finally, our observations remain compatible with prior expectations from the TRM framework itself, namely that CI can be highly beneficial, while also suggesting that its interaction with hierarchy may be especially valuable in domains whose symbolic-temporal structure aligns well with target-state options.

Overall, the empirical answer is therefore conditional rather than absolute. The hierarchical timed extension is clearly beneficial on Taxi, especially when combined with CI, but it is not uniformly superior across all evaluated domains.

10 Discussion

The results suggest that the contribution of the proposed extension should be understood in terms of interacting effects rather than as a single monolithic improvement. On Taxi, local counterfactual imagining already provides a strong gain over the vanilla baseline, which confirms the practical usefulness of counterfactual reuse in the timed setting. The hierarchy-only variant, by contrast, remains close to vanilla at the final step, which indicates that option decomposition alone is not always enough to produce a large gain. However, the combination of hierarchy and CI performs best overall, suggesting that these two mechanisms improve different aspects of the learning process and can reinforce each other when the task structure is favorable.

A natural interpretation is that Taxi offers a symbolic-temporal decomposition that aligns well with target-state options. In such a setting, the high-level choice of a TRM target can provide useful structure, while the low-level CI mechanism improves credit assignment and sample reuse inside each option. Frozen Lake appears less favorable in this respect. Its higher stochasticity may blur the advantage of symbolic-temporal decomposition, especially at the present sample budget, making it harder to observe a stable benefit from hierarchy.

This reading is consistent with the broader intuition behind HRM. Hierarchy can be highly beneficial when subgoals correspond to meaningful intermediate structure in the environment, but it should not be expected to dominate uniformly across all tasks. In the present study, Taxi provides a strong positive case for the proposed timed hierarchical extension, whereas Frozen Lake highlights the importance of task dependence and variance.

11 Threats to Validity and Limitations

Several limitations should be kept in mind when interpreting these results. First, the scope of the study is restricted to the digital-time setting; no real-time hierarchical extension is considered here. Second, the empirical evaluation is limited to two environments, Taxi and Frozen Lake, so broader benchmarking would be needed before drawing stronger general conclusions.

A further limitation concerns statistical reliability. In particular, the Frozen Lake experiments exhibit substantial variance, and final-step means alone are therefore insufficient for strong comparative claims. More extensive seed budgets, longer horizons, and formal statistical testing would provide a firmer basis for interpretation. Finally, some of the design choices adopted in this work, such as target-state option indexing and strictly local CI, are principled simplifications, but they remain modeling choices that may affect the observed outcomes.

12 Conclusion

We proposed and implemented a digital-time HRM extension for Timed Reward Machines, together with a combined HRM+CI variant, while preserving the original TRM product semantics and evaluation protocol.

Empirically, the extension is meaningful. On Taxi, *TRM+HRM+CI* achieves the best final performance in both discounted return and episode-time efficiency, and it also converges faster than the non-hierarchical baselines. On Frozen Lake, however, the results are much more variable and do not support a robust universal ranking under the present training budget.

The main conclusion is therefore one of conditional benefit. Hierarchical decomposition in digital TRMs appears promising, especially when combined with local CI and when the underlying task

structure aligns well with symbolic-temporal subgoals. At the same time, the current results also make clear that this benefit is not automatic and depends on the environment.

Natural next steps include increasing seed and horizon budgets, adding formal statistical tests, studying sensitivity to discretization choices, and eventually extending the hierarchical design to real-time TRM abstractions.

References

[1] R. Toro Icarte, T. Klassen, R. Valenzano, and S. McIlraith. Reward Machines: Exploiting Reward Function Structure in Reinforcement Learning. *Journal of Artificial Intelligence Research*, 2022.

[2] A. Majumdar, R. Raha, R. Roy, D. Parker, and M. Kwiatkowska. About Time: Model-free Reinforcement Learning with Timed Reward Machines.

[3] R. S. Sutton, D. Precup, and S. Singh. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112(1–2):181–211, 1999.